so we're expecting the cache to be busted after that. This means that the remaining steps will have to be built from scratch, including the bundle install, which will take some time.

```
$ docker build -t railsapp .
```

With our newly built image, let's see what happens when we modify our README.md file again. Go ahead and make any minor change to README.md, save the change, and then rebuild the image:

```
$ docker build -t railsapp .
Sending build context to Docker daemon  102.9kB
Step 1/7 : FROM ruby:2.6
 ---> f28a9e1d0449
Step 2/7 : RUN apt-get update -yqq && apt-get install -yqq --no-install-
recommends   nodejs
 ---> Using cache
 ---> 29c3dee2b8c7
Step 3/7 : COPY Gemfile* /usr/src/app/
 ---> Using cache
 ---> 050a87002be1
Step 4/7 : WORKDIR /usr/src/app
 ---> Using cache
 ---> d227daeedb1e
Step 5/7 : RUN bundle install
 ---> Using cache
 ---> 616b88058c4b
Step 6/7 : COPY . /usr/src/app/
 ---> b189758b9ded
Step 7/7 : CMD ["bin/rails", "s", "-b", "0.0.0.0"]
 ---> Running in fad4be04ab20
Removing intermediate container fad4be04ab20
 ---> 9be0cf184e64
Successfully built 9be0cf184e64
Successfully tagged railsapp:latest
```

That was much faster than when we changed the README.md earlier. This time, it didn't result in the gems being rebuilt. Lines 6–8 in the Dockerfile could use the cache because nothing had changed at that point. Docker just has to rebuild the final two steps, both of which are fast.

The Finishing Touch

Our Dockerfile is glorious, isn't it? A masterpiece. This little baby is going to get our Rails app development really going places. So let's do what all true artists do and sign our work.

Unlike a painter who signs the bottom right of their canvas, Docker aficionados typically assert their authorship of an image by setting a *maintainer label* as